

Maintaining Centdians in a Fully Dynamic Forest

Hung-Lung Wang*

Department of Computer Science and Information Engineering
National Taiwan University, Taipei, Taiwan 106

Abstract

In this paper, we consider the problem of maintaining the centdians in a fully dynamic forest. A centdian is a location in the given network which minimizes a convex combination of the longest distance and the sum of distances from all clients to it. A fully dynamic forest is a forest, i.e., a set of trees, in which the following operations are allowed: to insert an edge between different trees in the forest, to delete an edge in the forest, and to change a vertex weight in the forest. We propose an algorithm to maintain a centdian of each tree in the given fully dynamic forest. It preprocesses the input forest with time and space linear to the size of the forest and uses $O(\log n)$ update time per edge insertion, edge deletion, or vertex weight change, where n is the number of vertices of the trees involved in the update.

1 Introduction

Facility location problems have been widely investigated in the fields of operation research, communication, and theoretical computer science. In a facility location problem, there are two subsets of a given network which describe the locations of the customers and the locations where one can deploy the facilities, and one is asked to find a deployment of facilities on which a given objective function assumes its optimal value. Normally, a given network is modeled as a connected undirected graph, and, depending on the requirements, edge lengths or vertex weights could be imposed. Based on this model, a facility location problem can be categorized by $\Gamma/\Omega/p$ together with the objective function and the network type, where Γ , the supply set, is the set of locations where the facilities can be deployed, Ω , the demand set, is the set of customers' locations, and p is the number of facilities one is asked to deploy. The sets Γ and Ω

normally belong to $\{V(G), A(G)\}$, where G is the given network, $V(G)$ is the vertex set of G , and $A(G)$ denotes the continuous points on the edges of G .

Two classical problems are the center problem and the median problem. For the center problem, the objective is to minimize the longest distance from a customer to a nearest facility. For the median problem, the objective is to minimize the sum of distances from all customers to their nearest facilities. For the static case, there are many results on these two problems [5, 7, 12, 13, 14, 15]. For the dynamic case, Alstrup et al. [2, 3] showed that the centers and medians can be maintained in a fully dynamic forest with preprocessing time and space proportional to the size of the forest and $O(\log n)$ update time per edge insertion, deletion, or change to vertex weight, where n is the size of the trees involved in the update. For other previous results, see [2]. Since the center problem and the median problem are widely studied, intuitively one may have the question: "Why do we need two kinds of measurements?" There must be different requirements on the deployment of facilities. Thus, to find a balance between two different measurements is a natural extension. Here we consider the problem of maintaining the centdians in a fully dynamic forest. The centdian problem, which was proposed by Halpern [8, 9, 10], takes both of the longest distance and the average distance into consideration, and the objective function is defined to be a convex combination of that of the center problem and that of the median problem. Unlike the center problem and the median problem, there are not so many algorithmic results on the centdian problem [18]. Halpern showed that there is a "good" property of the objective function of the centdian problem similar to that of the center problem and that of the median problem, i.e. the convexity of the objective function. In this paper, we apply this property to maintaining the centdians in a fully dynamic forest.

In dealing with problems on (fully) dynamic

*r92085@csie.ntu.edu.tw

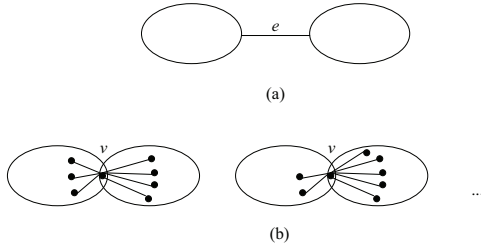


Figure 1: The difference between (a) a topology tree and (b) a top tree

forests, the top tree proposed by Alstrup et al. [1, 2, 3], the topology tree proposed by Frederickson [6], and the data structure proposed by Sleator and Tarjan [16] can be of great help. It has been shown that the above three data structures can be implemented by each other [3]. In this paper, we resolve the problem by using Frederickson's topology tree [6] together with the nonlocal search method proposed by Alstrup et al. [2, 3]. Even though the topology tree data structure is developed for trees with maximum degree three, there is some useful property on the topology tree data structure. A topology tree is built by clustering the vertices of the input tree, and two clusters are connected according to an edge. For a given tree T and an edge e , there is only one bipartition of T according to e , as shown in Figure 1(a). However, in a top tree, a cluster consists of edges, and two clusters are connected according to the vertex which is in both clusters. For a given tree T and a vertex v , there are many possible bipartitions according to v , as shown in Figure 1(b).

One can see that there are some “uncertainties” in the top tree data structure. In maintaining the dynamic centers and the dynamic medians, such an uncertainty can be resolved as shown in Lemma 6 and Corollary 9 in [2]. However, in maintaining the dynamic centdians, it takes more time to deal with such an uncertainty, and that is the reason why we use the topology tree. In Section 4, we will show how the topology tree can be a great help. However, in [6] the edge to be inserted in a given forest is restricted such that the maximum degree of the resulting forest will never exceed three. In Section 3, we give a modification to remove this restriction so that an edge insertion can be applied on an arbitrary vertex pair (u, v) , where u and v are in different trees in the given forest.

The rest of this paper is organized as follows. In Section 2, we define the problem and summarize the properties of centdians and topology trees. In Section 3, we give a transformation to the in-

put forest and modify the operation of inserting an edge from [6] so that the restriction of the inserted edge can be removed. In Section 4, a select function of the centdian problem is given so that the nonlocal search technique [3] can be applied. Finally, concluding remarks are given in Section 5.

2 Problem Definition and Preliminaries

In this section, we describe the notation and some results on the centdian problem and topology trees. They will be used in resolving the dynamic centdian problem. In this paper, the facility location problem we consider is of the category $V(T)/V(T)/1$, i.e. to maintain a centdian in each tree of the given forest, and for succinctness, we omit the term and call it the dynamic centdian problem directly.

2.1 Problem Definition and the Properties of a Centdian in a Tree

Let $G = (V(G), E(G), w, l)$ be a graph, where $V(G)$ denotes the vertex set of G , $E(G)$ denotes the edge set of G , $w : V(G) \mapsto \{x \mid x \in \mathbf{R}\}$ denotes the vertex weight, and $l : E(G) \mapsto \{x \mid x \in \mathbf{R} \text{ and } x > 0\}$ denotes the edge length. We also use $w(G)$ to denote $\sum_{v \in V(G)} w(v)$. The distance between two vertices u and v is defined to be the length of the shortest path $\mathcal{P}[u, v]$ between u and v , which is $\sum_{e \in E(\mathcal{P}[u, v])} l(e)$ and is denoted by $d_G(u, v)$. Moreover, each vertex v is associated with three values defined by the following three functions: (i) the center function $f_c^G(v) = \max_{u \in V(G)} d_G(u, v)$, (ii) the median function $f_m^G(v) = \sum_{u \in V(G)} w(u) \cdot d_G(u, v)$, and (iii) the centdian function $f_\theta^G(v) = \lambda f_m^G(v) + (1 - \lambda) f_c^G(v)$, where $0 < \lambda < 1$. The vertices where the three functions assume their minimum are the center, median, and centdian, respectively and are denoted by c_G , m_G , and θ_G , respectively. The degree of a vertex v in G is denoted by $\deg_G(v)$, and the maximum degree of G is denoted by $\Delta(G) = \max_{v \in V(G)} \deg_G(v)$.

Now we are ready to define the dynamic centdian problem.

Definition 1: (*the dynamic centdian problem*) Given is a forest $F = \{T_1, T_2, \dots, T_k\}$, where T_i denotes a connected component, i.e. a tree, in F , for $1 \leq i \leq k$. One is asked to maintain a centdian

of each tree in the forest after a *link*, a *cut*, or a *set_w* operation, where

- *link*(u, v) (or *link*(e)) : insert an edge $e = (u, v)$ into F , where $u \in V(T_i)$, $v \in V(T_j)$, and $i \neq j$.
- *cut*(e) : delete an edge $e \in E(F)$.
- *set_w*(v) : set $w(v)$ to be a given real for $v \in V(F)$.

A forest is *dynamic* if *link* and *cut* are allowed. While *set_w* is also allowed, the forest is *fully dynamic*. In Properties 1 and 2, we summarize the properties which we use to maintain the centdians in a fully dynamic forest.

Property 1: [8] Given a tree T , the following statement holds: θ_T is in $\mathcal{P}[m_T, c_T]$, and f_θ^T is convex on $\mathcal{P}[m_T, c_T]$.

Property 2: [19] Let T be a tree, and $T - \{e\} = T_1 \cup T_2$, where $e = (u, v) \in E(T)$, $u \in V(T_1)$, and T_1 and T_2 are subtrees of T . For centers, $c_T \in V(T_1)$ if and only if $f_c^T(u) \leq f_c^T(v)$. For medians, $m_T \in V(T_1)$ if and only if $w(T_1) \geq w(T_2)$.

2.2 Topology Trees

The topology tree used in this paper is based on the *restricted multilevel partition* [6], and a restricted multilevel partition with respect to a tree T is a hierarchical clustering method of $V(T)$. At level i of the hierarchy, it is a restricted partition with respect to the tree which takes the clusters in level $i - 1$ as the tree vertex. A cluster with respect to T is a subset of $V(T)$ on which T induces a connected subgraph. A cluster C contains the following information:

- *boundary vertices*: the vertices are adjacent in T to some vertices not in C , denoted by $\mathcal{B}(C)$.
- *tree degree*: the number of tree edges with exactly one end vertex in C .
- *adjacency*: the clusters which C is adjacent to.

Two clusters are adjacent if there is an edge of T with one end vertex in each of the clusters. The definitions of a restricted partition, a restricted multilevel partition, and a topology tree based on the restricted multilevel partition in [6] are given as follows.

Definition 2: [6] (*restricted partition of order z*) Given a tree T with $\Delta(T) \leq 3$, a partition of $V(T)$ is called a restricted partition of order z respect to T if the following conditions hold:

- Each set in the partition is a vertex cluster of tree degree at most three.
- Each cluster of tree degree 3 is of cardinality 1.
- Each cluster of tree degree less than 3 is of cardinality at most z .
- No two adjacent clusters can be combined and still satisfy the above.

Definition 3: [6] (*restricted multilevel partition*) Given a tree T with $\Delta(T) \leq 3$, a restricted multilevel partition with respect to T is a set of partitions of $V(T)$ such that the following conditions hold:

- For each level $l = 0, 1, \dots, q$, the vertex clusters at level l form a partition of V .
- The clusters at level 0 form a restricted partition of order z .
- The clusters at any level $l > 0$ constitute a restricted partition of order 2 with respect to the tree resulting from viewing each cluster at level $l - 1$ as a vertex.
- There is precisely one vertex cluster at level q , which contains all vertices.

Definition 4: [6] (*topology tree*) Given a tree T with $\Delta(T) \leq 3$ and a restricted multilevel partition $\mathcal{R} = \bigcup_{i=0}^q R_i$ of T , where R_i denotes a restricted partition at level i , a topology tree $\mathcal{T}$ for T is a binary tree with $V(\mathcal{T}) = \{C \mid C \in \mathcal{R}\}$, $E(\mathcal{T}) = \{(C, C') \mid C \in R_i, C' \in R_{i-1}, \text{ and } \exists C'' \in R_{i-1} \text{ such that } C' \cup C'' = C, \text{ for } 1 \leq i \leq q\}$, and root $C_r \in R_q$.

In the rest of this paper, n denotes the size of the trees involved in an operation. Some properties of a topology tree are summarized in Property 3.

Property 3: Given a tree T with $\Delta(T) \leq 3$, a topology tree $\mathcal{T}$ of T based on the restricted multilevel partition can be constructed in linear time using linear space, and the height of $\mathcal{T}$ is $O(\log |V(T)|)$.

Proof: The result follows from Lemma 2.2 and Lemma 2.3 in [6]. $\square$

In [6], the given tree is updated by a $swap(e, f)$ operation, which replaces a tree edge e by a non-tree edge f and yields another tree. The $swap(e, f)$ operation can be viewed as the concatenation of $cut(f)$ and $link(e)$. Thus, by Lemma 3.1 of [6] we have the following property.

Property 4 : [6] Given a forest $F = \{T_1, T_2, \dots, T_k\}$ with $\Delta(F) \leq 3$ and topology trees for all T_i , where $1 \leq i \leq k$, the operations $link$ and cut can be maintained in $O(z + \log n)$ time if $link$ does not increase $\Delta(F)$ to be greater than three.

3 The Tree Transformation and the Modification of Topology Trees

In [6], the topology tree is constructed for a spanning tree T of a graph G with $\Delta(G) \leq 3$. Moreover, the edge to be inserted is restricted to be in G so that the maximum degree of T is no more than three after an edge insertion, and thus the topology tree can be maintained correctly. In the following, we set the order of the restricted partition at level 0 to be one and show how to remove the restriction on an edge insertion, i.e. to make the $link$ operation works correctly. Our strategy works as follows: first, the input forest is transformed into a forest with maximum degree no more than three by dividing vertices with degree greater than three so that the topology tree can be constructed. If the maximum degree of the transformed forest is greater than three after a $link$ operation, we divide the vertices with degree greater than three and then reconstruct the topology tree. For the operation cut , if an end vertex of the deleted edge is generated by a $link$ operation, we need to contract it with another vertex so as to prevent the unbounded increase on the size of the data structure. If both end vertices of the deleted edge are in the input tree, the modification is the same as in [6]. The details for maintaining set_w will be discussed later.

Let $F = \{T_1, T_2, \dots, T_k\}$ be the input forest. We construct a forest $F' = \{T'_1, T'_2, \dots, T'_k\}$ from F by the procedure VERTEXDIVIDE. For each $T_i \in F$, we apply VERTEXDIVIDE iteratively until there is no vertex with degree greater than three. The procedure works as follows: for a vertex v with

$deg_T(v) \leq 3$, it does not do anything; for a vertex v with $deg_T(v) = s > 3$ and with neighbors $\{u_0, u_1, \dots, u_{s-1}\}$, it deletes the edges (v, u_i) for $0 \leq i < s$, divides v into $\{v_1, v_2, \dots, v_{s-2}\}$, marks v_i as a divided vertex for $1 \leq i \leq s-2$, and adds edges (u_0, v_1) , (u_{s-1}, v_{s-2}) , (u_i, v_i) for $1 \leq i \leq s-1$, and (v_i, v_{i+1}) for $1 \leq i \leq s-3$. The edge length function l' and vertex weight function w' of the transformed forest F' are defined as follows. If a vertex v is divided, then $l'(u_i, v_j) = l(u_i, v)$ for $0 \leq i \leq s-1$ and $1 \leq j \leq s-2$, $l'(v_i, v_{i+1}) = \epsilon > 0$ for $1 \leq i \leq s-3$, $w'(v_i) = 0$ for $2 \leq i \leq s-2$, and $w'(v_1) = w(v)$. Otherwise, the edge lengths and vertex weights keep the same. The value ϵ denotes a sufficiently small value which can be treated as zero during the implementation. In maintaining the dynamic centdian, theoretically it is required that $\epsilon > 0$ because the convexity of the centdian function holds only when the edge lengths are positive.

In Lemmas 5, 6, and 7, we introduce the relationship between F and F' . It is clear that the number of components in F' is the same as that of F since no component in F will be disconnected or be connected with another one via VERTEXDIVIDE. Therefore, for each tree in F there is a corresponding tree in F' . Let T be a tree in F and T' be its corresponding tree in F' . For each $v \in V(T)$, there is a vertex $v' \in V(T')$ which is either v itself or is divided from v . We call v and v' the corresponding vertices with each other.

Lemma 5: T' is a tree.

Proof: Since T' results from iteratively dividing vertices with degree greater than three in T , it suffices to show that the tree structure keeps after each iteration. If a vertex is not divided, the tree structure keeps. If a vertex v is divided, each branch of v connects to exactly one vertex of the path $(v_1, v_2, \dots, v_{s-2})$. Thus, the resulting graph is connected and with no cycle, i.e. a tree. $\square$

Lemma 6: For u and v in T , let u' and v' in T' be the corresponding vertices of u and v , respectively. $\mathcal{P}[u', v']$ consists of the corresponding vertices of the vertices in $\mathcal{P}[u, v]$.

Proof: It suffices to claim that at any iteration, each path $\mathcal{P}[u'', v'']$ in the partially transformed tree T'' (Lemma 5) consists of the corresponding vertices of the vertices in $\mathcal{P}[u, v]$. We prove by induction on the number of processed vertices. In the beginning, the claim holds true since no

vertex is divided. Suppose the claim holds true before the i th iteration. If there is no vertex divided at the i th iteration, the claim follows. If a vertex v is divided, for each path in each branch of v the claim holds by the induction hypothesis. For a path passing through v_i , a corresponding vertex of v , in T'' , it can be partitioned into two parts: (i) the part which is in the branches of v_i , and (ii) the part consists of the corresponding vertices of v . For (i) the claim follows by the induction hypothesis. For (ii) the claim follows because there are only corresponding vertices of v . Therefore, the claim holds after iteration i , and by mathematical induction the lemma follows. $\square$

Lemma 7: For all $u, v \in V(T)$, as $\epsilon \rightarrow 0$, $d_T(u, v) = d_{T'}(u', v')$ for all corresponding vertices u' and v' of u and v , respectively.

Proof: By Lemma 6, one can see that $d_{T'}(u', v') = d_T(u, v) + r\epsilon$ for some finite number r . Thus, $d_{T'}(u, v) = d_T(u, v)$ as $\epsilon \rightarrow 0$. $\square$

Corollary 8: As $\epsilon \rightarrow 0$, a vertex v is a centdian of T if and only if there exists an i , $1 \leq i \leq s-2$, such that v_i is a centdian of T' .

To use the topology tree to resolve the dynamic centdian problem, we need to record some additional information when applying the restricted multilevel partition. In the following, we describe the additional information we need and show how efficiently the topology tree can be build.

Besides the information contained in a cluster introduced in Section 2, the cluster C we use contains the following information:

- $\max d(v, C)$: the longest distance from v in C , where $v \in \mathcal{B}(C)$.
- $d\mathcal{B}(C)$: if $|\mathcal{B}(C)| = 2$, it denotes the distance between the boundary vertices. Otherwise, it is zero.
- $w(C)$: $\sum_{v \in C} w(v)$.

For a cluster C_0 at level 0, it consists of only one vertex, say v , and $\mathcal{B}(C_0) = \{v\}$, $\max d(v, C_0) = 0$, $d\mathcal{B}(C_0) = 0$, and $w(C_0) = w(v)$. When constructing a cluster C_i at level i , there are two cases: (i) C_i consists of only one cluster C_{i-1} at level $i-1$ and (ii) C_i consists of two clusters C_{i-1} and C'_{i-1} at level $i-1$. In (i), all the information C_i needs inherits directly from C_{i-1} . In (ii), let

$\mathcal{B}(C_{i-1}) = \{u_{i-1}, v_{i-1}\}$, $\mathcal{B}(C'_{i-1}) = \{u'_{i-1}, v'_{i-1}\}$, and $(u_{i-1}, u'_{i-1}) \in E(F)$ (note that u_{i-1} and u'_{i-1} may be the same as v_{i-1} and v'_{i-1} , respectively). It is obvious that $\mathcal{B}(C) = \{v_{i-1}, v'_{i-1}\}$, and therefore

$$\begin{aligned} \max d(v_{i-1}, C_i) &= \max\{\max d(v_{i-1}, C_{i-1}), \\ &\quad d\mathcal{B}(C_{i-1}) + \max d(u'_{i-1}, C'_{i-1}) \\ &\quad + l(u_{i-1}, u'_{i-1})\}, \end{aligned}$$

$$\begin{aligned} \max d(v'_{i-1}, C_i) &= \max\{\max d(v'_{i-1}, C'_{i-1}), \\ &\quad d\mathcal{B}(C'_{i-1}) + \max d(u_{i-1}, C_{i-1}) \\ &\quad + l(u_{i-1}, u'_{i-1})\}, \end{aligned}$$

$$d\mathcal{B}(C) = d\mathcal{B}(C_{i-1}) + d\mathcal{B}(C'_{i-1}) + l(u_{i-1}, u'_{i-1}),$$

and $w(C_i) = w(C_{i-1}) + w(C'_{i-1})$. From the above analysis, we have the following lemma.

Lemma 9: For a restricted multilevel partition, given the restricted partition at level $i-1$, each cluster at level i can be constructed in constant time.

Therefore, the topology tree of T with the information we need can still be build in $O(|V(T)|)$ time and space and with height $O(\log |V(T)|)$.

Now we discuss how the topology tree can support the *link*, *cut*, and *set.w* operations. For *set.w*, the only changed clusters are on the path from the leaf representing the changed vertex v to the root of the topology tree. Therefore, a bottom up procedure on this path is applied. For all clusters C on this leaf to root path, $w(C)$ is changed to be $w(C) + w' - w(v)$, where w' is the new weight of v . The time for *set.w* is $O(\log n)$ since it takes constant time to process each cluster on the leaf to root path.

For *link*, let (u, v) be the inserted edge for vertices u and v in F . We link the corresponding vertices u_1 and v_1 in F' , the forest transformed by VERTEXDIVIDE. If $\Delta(F') \leq 3$ after a *link* operation, the result follows directly from Property 4. As $\Delta(F') > 3$ after a *link* operation, we apply VERTEXDIVIDE on the vertices with maximum degree in F' , and the resulting forest F'' has $\Delta(F'') \leq 3$. Since the forest is always transformed into a forest with maximum degree no more than three, the only case which needs modification is when $\Delta(F') = 4$ after a *link* operation. There

are at most two vertices with degree four after a *link* operation, i.e. the end vertices of the inserted edge. Through VERTEXDIVIDE, the vertex with degree four will be divided into two vertices with degree three. Similar to Corollary 8, there is a correspondence between the centdians of F and F'' . Since there are at most two more base clusters build after *link*, the topology trees for F'' can be maintained according to the argument in Lemma 2.1 of [6].

For *cut*, let (u, v) be the deleted edge in F' . If, without loss of generality, u is marked as a divided vertex, there must be an edge (u', u) with length ϵ , and u' is also a marked vertex. One can see that after the deletion of (u, v) , the degree of u becomes two. We contract (u, u') to be a vertex u'' and make the edges which are incident to u and u' incident to u'' . If there is no edge incident to u'' with length ϵ , we unmark u'' . Otherwise, u'' is still marked as a divided vertex. Similar to the argument in Lemma 2.1 of [6], the topology tree can be maintained with the *cut* operation. According to the contraction procedure, the size of the topology tree will be always proportional to the number of vertices in the input forest.

Based on the above analysis, we have the following lemma.

Lemma 10: Given the topology trees for a forest, each of the operations *link*, *cut*, and *set_w* can be maintained in $O(\log n)$ time.

4 Maintaining the Centdians in a Fully Dynamic Forest

In this section, we describe how to maintain the centdians in a fully dynamic forest. We use the nonlocal search method proposed by Alstrup et al. [2, 3]. The nonlocal search method can be applied to maintain the information with nonlocal properties. Following the terminology in [2, 3], a property of an object v is local if it holds true in all subgraphs where v is. For example, being a longest path is local, but being a centdian is not. The nonlocal search method is based on the following theorem.

Theorem 11: [2, 3] Given a top tree, after $O(\log n)$ calls to *select*, *merge*, and *split*, there is a unique edge (u, v) contained in all clusters chosen by *select*, and then (u, v) is returned.

Although the nonlocal search method is developed based on top trees, it can be applied on topology trees because these two data structures are

constructed in a similar way. For each of the operations *merge*, *split*, and *select* in Theorem 11, there is a corresponding operation with respect to dealing with nonlocal search on topology trees, and we call them MERGE, SPLIT, and SELECT, respectively. The MERGE and SPLIT are the two operations for maintaining a topology tree, i.e. to merge two clusters at level $i - 1$ to be a cluster at level i , and to split a cluster at level i into two clusters at level $i - 1$, respectively. The function SELECT takes the root of a topology tree as the argument and output the child of the root cluster which contains the object we want. For the detailed implementation, please refer to [2, 3].

In the dynamic centdian problem, as shown in Lemma 9, MERGE and SPLIT can be done in constant time. To apply Theorem 11, it suffices to develop a function SELECT which can choose a child cluster for a given root of a topology tree efficiently such that the chosen cluster contains a centdian.

Lemma 12: Let $T = (V(T), E(T), w, l)$ be a tree, and $T - \{e\} = T_1 \cup T_2$, where $e = (u, v) \in E(T)$, $u \in V(T_1)$, and T_1 and T_2 are subtrees of T . A centdian is in T_1 if and only if one of the following conditions holds: (i) $w(T_1) \geq w(T_2)$ and $f_c^T(v) \geq f_c^T(u)$, (ii) $w(T_1) > w(T_2)$, $f_c^T(u) > f_c^T(v)$, and $w(T_1) - w(T_2) \geq \frac{1-\lambda}{\lambda}$, and (iii) $w(T_2) > w(T_1)$, $f_c^T(v) > f_c^T(u)$, and $w(T_2) - w(T_1) \leq \frac{1-\lambda}{\lambda}$.

Proof: For sufficiency, if condition (i) holds, by Property 2 we have that c_T and m_T belong to T_1 , and by Property 1, the centdian of T is in T_1 . If condition (ii) holds, by Property 2 we have $m_T \in V(T_1)$ and $c_T \in V(T_2)$. Therefore, u and v are in $\mathcal{P}[c_T, m_T]$. From $w(T_1) - w(T_2) \geq \frac{1-\lambda}{\lambda}$, we have

$$\begin{aligned} \lambda(w(T_1) - w(T_2)) &\geq 1 - \lambda \\ \Rightarrow \lambda(w(T_1) - w(T_2)) \cdot d_T(u, v) &\geq (1 - \lambda) \cdot d_T(u, v) \\ \Rightarrow \lambda(f_m^T(v) - f_m^T(u)) &\geq (1 - \lambda)(f_c^T(u) - f_c^T(v)) \\ \Rightarrow f_\theta^T(v) &\geq f_\theta^T(u). \end{aligned}$$

By Property 1, the centdian of T is in T_1 . A similar argument holds for condition (iii).

For necessity, if a centdian of T is in T_1 , there are three cases: either (a) $\mathcal{P}[m_T, c_T] \subseteq T_1$, (b) $c_T \in V(T_1)$ and $m_T \in V(T_2)$, or (c) $c_T \in V(T_2)$ and $m_T \in V(T_1)$. In case (a), by Property 2 condition (i) follows. In case (b), u and v are in $\mathcal{P}[m_T, c_T]$. By Property 2, we have $w(T_1) \leq w(T_2)$ and $f_c^T(u) \leq f_c^T(v)$. If $w(T_1) = w(T_2)$ and $f_c^T(u) < f_c^T(v)$, condition (i) follows. If $w(T_1) < w(T_2)$ and $f_c^T(u) = f_c^T(v)$, by Property 1,

the centdian of T cannot be in T_1 . Thus, we suppose that $w(T_1) < w(T_2)$ and $f_c^T(u) < f_c^T(v)$. For the centdian functions $f_\theta^T(u)$ and $f_\theta^T(v)$, by Property 1 we have

$$\begin{aligned} 0 &\geq f_\theta^T(u) - f_\theta^T(v) \\ &= \lambda(f_m^T(u) - f_m^T(v)) + (1 - \lambda)(f_c^T(u) - f_c^T(v)) \\ &= \lambda(w(T_2) - w(T_1)) \cdot d_T(u, v) - (1 - \lambda) \cdot d_T(u, v), \end{aligned}$$

which implies $w(T_2) - w(T_1) \leq \frac{1-\lambda}{\lambda}$, and condition (iii) follows. For case (c), the argument is similar to that in case (b), and condition (ii) follows. $\square$

By Lemma 12, we define the function SELECT as follows. Given a root cluster of a topology tree with children A and B , which correspond to T_1 and T_2 , respectively as in Lemma 12, let $e = (u, v)$, $u \in A$ and $v \in B$, be the edge which connects A and B . If one of the conditions in Lemma 12 holds, then A is selected. Otherwise, cluster B is selected. By Theorem 11 and Lemma 12, after $O(\log n)$ calls to the select function, there is exactly one vertex which is in all clusters chosen by SELECT, and it is the centdian of the tree in the forest. Since it takes constant time to perform MERGE, SPLIT, and SELECT, the nonlocal search procedure takes $O(\log n)$ time. We conclude this section by the following theorem.

Theorem 13: Given the topology trees of the trees in a forest F , it takes $O(\log n)$ time to maintain the centdians in F per *link*, *cut*, or *set_w* operation.

5 Concluding Remarks

Maintaining facilities in a dynamic network is an interesting topic. The center, median, and centdian of a tree can be maintained efficiently because of the convexity of their objective functions. An intuitive extension is to consider the more general form of the centdian function, i.e. to consider the weighted distance in the center function. Similarly, to maintain obnoxious centers [4, 17] in a dynamic network is also an interesting problem. All of these will be the future work.

Acknowledgments. I thank my advisor Prof. Kun-Mao Chao for valuable comments. Hung-Lung Wang was supported in part by NSC grants 95-2221-E-002-126-MY3 and 96-2221-E-002-034 from the National Science Council, Taiwan.

References

- [1] S. Alstrup, J. Holm, K. de Lichtenberg, M. Thorup, Minimizing diameters of dynamic trees, *International Colloquium on Automata, Languages, and Programming*, 270–280, 1997.
- [2] S. Alstrup, J. Holm, and M. Thorup, Maintaining center and median in dynamic trees, *Scandinavian Workshop on Algorithm Theory*, pp. 46–56, 2000.
- [3] S. Alstrup, J. Holm, K. de Lichtenberg, M. Thorup, Maintaining information in fully dynamic trees with top trees, *ACM Transactions on Algorithms* 1: 2, pp. 243–264, 2005.
- [4] R. E. Bruckard, H. Dollant, Y. Lin, and G. Rote, The obnoxious center problem on a tree, *SIAM Journal on Discrete Mathematics*, 14: 4, pp. 498–509, 2001.
- [5] G. N. Frederickson, Parametric search and locating supply centers in trees, *Workshop on Algorithms and Data Structures*, pp. 299–319, 1991.
- [6] G. N. Frederickson, Ambivalent data structures for dynamic 2-edge-connectivity and k smallest spanning trees, *SIAM Journal on Computing* 26: 2, pp. 484–538, 1997.
- [7] B. Gavish, S. Shridhar, Computing the 2-median on tree networks in $O(n \log n)$ time, *Networks*, 26, pp. 305–317, 1995.
- [8] J. Halpern, The location of a center-median convex combination on an undirected tree, *Journal of Regional Science*, 16: 2, pp. 237–245, 1976.
- [9] J. Halpern, Finding minimal center-median convex combination (cent-dian) of a graph, *Management Science*, 24: 5, pp. 535–544, 1978.
- [10] J. Halpern, Duality in the cent-dian of a graph, *Operations Research*, 28, pp. 722–735, 1980.
- [11] J. Holm, K. de Lichtenberg, Top-trees and dynamic graph algorithms, Technical Report DIKU-TR-98/17, Department of Computer Science, University of Copenhagen, 1998.
- [12] O. Kariv and S. L. Hakimi, An algorithmic approach to network location problems, Part I: The p -centers, *SIAM Journal on Applied Mathematics*, 37: 2, pp. 441–461, 1979.

- [13] O. Kariv and S. L. Hakimi, An algorithmic approach to network location problems, Part II: The p -medians, *SIAM Journal on Applied Mathematics*, 37: 3, pp. 539–560, 1979.
- [14] N. Megiddo, A. Tamir, E. Zemel and R. Chandrasekaran, An $O(n \log^2 n)$ algorithms for the k th longest path in a tree with applications to location problems, *SIAM Journal on Computing*, 10: 2, pp. 328–337, 1981.
- [15] G. Proietti, P. Widmayer, Partitioning the nodes of a graph to minimize the sum of subgraph radii, *International Symposium on Algorithms and Computation*, pp. 578–587, 2006.
- [16] D. D. Sleator, R. E. Tarjan, A data structure for dynamic trees, *Journal of Computer and System Sciences* 26: 3, pp. 362–391, 1983.
- [17] A. Tamir, Improved complexity bounds for center location problems on networks by using dynamic data structures, *SIAM Journal on Discrete Mathematics*, 1, pp. 377–396, 1988.
- [18] A. Tamir, D. Pérez-Brito, and J. A. Moreno-Pérez, A polynomial algorithm for the p -centdian problem on a tree, *Networks*, 32, pp. 255–262, 1998.
- [19] B. Y. Wu and K. M. Chao, *Spanning Trees and Optimization Problems*, Chapman and Hall/CRC Press, USA, 2004.